AI-Powered Fake News Detection Using Text Classification

Department of Computer Science and Engineering

Generated 29 September 2026 by `python -m src.report from the artifacts in results/`.

ABSTRACT

Fabricated news spreads faster than verified news and is impossible to screen by hand at newsroom speed. This report presents a supervised text-classification pipeline that labels an article as authentic or fabricated and returns a confidence together with the terms that drove the decision. The pipeline compares four algorithms -- K-Nearest Neighbours, Logistic Regression, Random Forest and a multi-layer perceptron -- over three feature representations: Bag-of-Words, TF-IDF and latent-semantic embeddings fitted by truncated SVD. Vocabulary, IDF weights and the embedding basis are fitted on the training split only, and the deployed pair is chosen on a held-out validation split, so the hold-out is scored exactly once. The selected combination, Bag-of-Words with Random Forest, reaches an accuracy of 0.9950, a precision of 1.0000, a recall of 0.9914 and an F1 score of 0.9957 on a stratified hold-out of 400 articles. The reported confidence is the model's own score, not a calibrated probability.

Index Terms -- Fake news detection, text classification, natural language processing, TF-IDF, latent semantic analysis, random forest, Streamlit.

I. INTRODUCTION

Artificial intelligence is the broadest term: it describes any machine that performs tasks normally requiring human intelligence. Fake-news detection is a narrow, concrete instance of it. A manual fact-checker must read an article, recall what is already known about the claim, judge whether the framing is honest, and then write a verdict. That is a *reasoning* task, and doing it at the speed of the news cycle is impossible by hand. An AI system performs the same decision function automatically: it ingests the article, produces a judgement, and returns it in milliseconds, so every article can be screened rather than only the ones a human had time to reach.

Problem statement. Given a news article, decide whether it is authentic reporting or fabricated content, and do it fast enough to screen the whole incoming feed. The hard part is that the two classes are linguistically similar: both are written in formal register, both quote sources, and both cover the same events. The difference lies in statistical regularities of vocabulary, sentence rhythm and sourcing convention that a human expert detects only slowly and inconsistently.

Why it matters. Fabricated news travels faster than verified news during elections, pandemics and conflicts, and it moves markets, changes health behaviour and erodes trust in journalism at exactly the moments that information matters most. Automating the screening step does not replace a fact-checker; it decides what a fact-checker should look at first.

This report evaluates four supervised classifiers -- K-Nearest Neighbours, Logistic Regression, Random Forest and a multi-layer perceptron -- over three feature representations: Bag-of-Words, TF-IDF and latent-semantic embeddings. On a stratified hold-out of 400 articles the selected combination, Bag-of-Words with Random Forest, reaches an accuracy of 99.50%, a precision of 100.00%, a recall of 99.14% and an F1 score of 99.57%.

All figures are computed from a corpus of 1,999 labelled articles (836 authentic, 1,163 fabricated).

II. DATASET DESCRIPTION

Source. The corpus is the public *Fake and Real News Dataset* [1], distributed through Kaggle. It ships as two files -- Fake.csv and True.csv -- each with a title, a text body, a subject desk and a date. A seeded, stratified 2,000-row sample of that dataset is committed to the repository as data/sample_dataset.csv so the project reproduces without a 117 MB download. The loader prefers the two full-size files whenever they are present in data/, so no code change is needed to scale up.

Table 1 -- Corpus composition

Property	*Value*
Articles	*1,999*
Authentic (label 0)	*836*
Fabricated (label 1)	*1,163*
Authentic share	*41.82%*
Fabricated share	*58.18%*
Mean characters per article	*2436.7*
Median characters per article	*2214.0*
Mean words per article	*400.6*
Median words per article	*367.5*

Longest article (characters)	36,088
Duplicate headline+body pairs	0
Rows with an empty body	0
Rows empty after cleaning	2

Table 2 -- Articles per news desk

Subject	Articles
politics	579
News	456
politicsNews	446
worldnews	390
US_News	59
Government News	41
left-news	28

> 35,142 distinct tokens survive cleaning across the corpus.

Columns

- title -- the headline.
- text -- the article body.
- subject -- the news desk, carried for exploration only and never fed to the classifier.
- label -- 0 for authentic, 1 for fabricated.
- full_text -- title + ". " + text, the string the model actually reads.
- Five engineered editorial signals (clickbait, emotion, sentiment, readability, source trust) computed for auditability.
- clean_text -- full_text after the NLP cleaning chain.

Class balance and data quality. The corpus is majority-fabricated, so accuracy alone would be a weak yardstick: a model that predicted FAKE for every article would already score 0.58. Precision, recall and F1 are reported alongside it, and no cost-sensitive reweighting is applied -- the classes are left as they occur naturally, so the reported figures describe the operating point a deployed screener would actually run at rather than a tuned one [10].

De-duplication. Duplicate headline-plus-body pairs are removed during preparation. The same article appearing on both sides of the split would inflate every score, and it is the single most common way a text classification benchmark produces a flattering and meaningless number.

III. METHODOLOGY

The pipeline is six runnable stages, each writing its artifacts to a declared path in src/config.py so the training scripts and the application share one source of truth.

Pipeline stages

- Acquire -- read the source corpus, normalise the schema to title / text / subject / label, drop duplicates.
- Clean -- apply the NLP preprocessing chain to full_text.
- Split -- draw a stratified hold-out *before* any representation is fitted.
- Represent -- fit Bag-of-Words, TF-IDF and the latent-semantic embedding on the training documents only.
- Select and train -- choose the combination with the best F1 on a validation split carved from the training data, then score every candidate once on the untouched hold-out.
- Serve -- the application reloads the fitted representation and the model, and returns class probabilities with a term attribution.

Text preprocessing

- Missing-value handling -- A null headline or body is replaced with an empty string before anything else runs, so no null can ever reach the vectoriser. A row that survives cleaning with no tokens at all is dropped when the split is drawn.
- Lowercasing -- Collapses casing variants so 'Government' and 'government' become one token and the vocabulary stops inflating.
- URL, HTML and digit removal -- Boilerplate carries no topical signal but is highly frequent, so removing it frees capacity for terms that discriminate.
- Punctuation removal -- Text is folded to ASCII before the 32 ASCII punctuation marks are stripped, so typographic apostrophes and dashes cannot fragment a word into two different tokens.
- Tokenisation -- The cleaned string is split on whitespace into individual terms.
- Stopword removal -- Function words carry grammatical structure but almost no class information; dropping them shrinks the vocabulary and raises the information density of what remains.
- Lemmatisation -- Inflected forms are mapped to a shared dictionary root, so 'reporting', 'reports' and 'reported' occupy one column instead of three.

Feature extraction

- Bag-of-Words -- CountVectorizer -- each document becomes a row of raw term counts over unigrams and bigrams, capped at 5,000 columns with min_df=2. Simple, fast and fully interpretable, but a term repeated fifty times outweighs a term repeated once

regardless of how rare it is, so common boilerplate dominates the geometry of the space.

- *TF-IDF -- TfidfVectorizer [2] -- the same counts re-weighted by inverse document frequency, with sublinear term frequency. A word that appears in most articles is scaled down; a word that appears in a handful is scaled up. This is the standard text representation and the reason K-Nearest Neighbours scores far better on it than on raw counts.*
- *Embeddings (LSA) -- The TF-IDF matrix is compressed with truncated singular value decomposition [3], the numerical core of latent semantic analysis [4], into 200 orthogonal latent axes and L2-normalised, giving one dense vector per article. Documents about the same subject end up close together even when they share no words, which is the defining property of a sentence embedding -- obtained here without a GPU, a pretrained checkpoint or an extra dependency. Each axis is labelled by the terms that load most heavily on it, so the dimensions stay interpretable.*

*Train/test protocol. The corpus is split 80/20 with train_test_split(test_size=0.20, random_state=42, stratify=y), and the split is materialised on disk *before* any vectoriser exists. The vocabulary, the IDF table and the embedding basis are therefore fitted on training documents only, and the hold-out is transformed by the already-fitted object. Model selection runs on a further stratified 20% validation split drawn from the training side, so the hold-out is scored exactly once and the reported figures are an estimate of generalisation rather than the maximum of twelve attempts. The comparison is evaluated with the metric definitions of [12] and the resampling protocol of [13], implemented in scikit-learn [7].*

Algorithms

- *K-Nearest Neighbours -- KNeighborsClassifier(n_neighbors=5). Instance-based and non-parametric; no training step, prediction cost scales with the stored corpus.*
- *Logistic Regression -- LogisticRegression(max_iter=2000, random_state=42). Parametric linear model over the feature vector.*
- *Random Forest [5] -- RandomForestClassifier(n_estimators=150, min_samples_leaf=2, random_state=42). Bagged decision trees; the selection in this run.*
- *Neural Network [6] -- MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=600, early_stopping=True, random_state=42). A simple feed-forward network trained by back-propagation.*

Selection is by F1 score, computed on a stratified validation split carved out of the training documents. The hold-out split is scored exactly once, after the choice is made, so the reported numbers are an estimate of generalisation rather than the maximum of twelve attempts. F1 is the criterion because it is the only headline metric that weighs precision and recall together, and because a missed fabricated article is generally more costly than a false alarm.

Deployment. The selected model and its fitted representation are persisted with joblib and served by a Streamlit application [9]. The inference path calls the same clean_text function the training path used, so the text the model sees at prediction time is the text it was trained on. The interface between the two is deliberately narrow -- a fitted object exposing transform(list_of_strings) -- which is what allows the application to serve a Bag-of-Words, TF-IDF or embedding pipeline without any change to the UI.

Preprocessing implementation. The cleaning chain is built on NLTK [8] for tokenisation, stopword lists and lemmatisation, with an offline fallback to the regular-expression patterns and static word lists in src/nlp_resources.py so the pipeline still runs on a machine with no corpora downloaded.

Table 3 -- Split composition

Split	Authentic	Fabricated	Total
Training	669	928	1,597
Hold-out	167	233	400

IV. RESULTS

Table 4 -- Model selection, by F1 on a 20% validation split of 320 training documents. The hold-out is not consulted here.

Representation	Model	Validation F1	Validation accuracy
Bag-of-Words	Random Forest	0.9973	0.9969
TF-IDF	Logistic Regression	0.9946	0.9938
TF-IDF	Random Forest	0.9946	0.9938
Bag-of-Words	Logistic Regression	0.9864	0.9844
Embeddings (LSA)	Neural Network	0.9864	0.9844
TF-IDF	Neural Network	0.9840	0.9812
Embeddings (LSA)	Logistic Regression	0.9836	0.9812
Bag-of-Words	Neural Network	0.9781	0.9750
Embeddings (LSA)	Random Forest	0.9468	0.9406
Embeddings (LSA)	K-Nearest Neighbors	0.8915	0.8844
TF-IDF	K-Nearest Neighbors	0.8902	0.8812
Bag-of-Words	K-Nearest Neighbors	0.8134	0.7562

Table 5 -- Required metrics on the hold-out split, computed from real predictions. Each algorithm is quoted on the representation that scored best for it; the deployed pair is the one selected in Table 4.

Model	Family	Representation	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest	Non-parametric (ensemble)	Bag-of-Words	0.9950	1.0000	0.9914	0.9957	0.9998
Logistic Regression	Parametric	Bag-of-Words	0.9925	1.0000	0.9871	0.9935	0.9994
Neural Network	Parametric (discriminative)	Embeddings (LSA)	0.9825	0.9829	0.9871	0.9850	0.9993
K-Nearest Neighbors	Non-parametric	Embeddings (LSA)	0.9025	0.9491	0.8798	0.9131	0.9662

Table 6 -- Full representation x algorithm grid (3 x 4)

Representation	Model	Accuracy	Precision	Recall	F1	Selected
Bag-of-Words	Logistic Regression	0.9925	1.0000	0.9871	0.9935	
Bag-of-Words	Random Forest	0.9950	1.0000	0.9914	0.9957	yes
Bag-of-Words	Neural Network	0.9800	0.9787	0.9871	0.9829	
Bag-of-Words	K-Nearest Neighbors	0.7225	0.6906	0.9485	0.7993	
TF-IDF	Logistic Regression	0.9850	0.9789	0.9957	0.9872	
TF-IDF	Random Forest	0.9925	1.0000	0.9871	0.9935	
TF-IDF	Neural Network	0.9800	0.9787	0.9871	0.9829	
TF-IDF	K-Nearest Neighbors	0.8900	0.9610	0.8455	0.8995	
Embeddings (LSA)	Logistic Regression	0.9850	0.9871	0.9871	0.9871	
Embeddings (LSA)	Random Forest	0.9350	0.9683	0.9185	0.9427	
Embeddings (LSA)	Neural Network	0.9825	0.9829	0.9871	0.9850	
Embeddings (LSA)	K-Nearest Neighbors	0.9025	0.9491	0.8798	0.9131	

Table 7 -- Confusion matrix per algorithm on the REAL / FAKE hold-out split

Model	Representation	REAL to REAL	REAL to FAKE	FAKE to REAL	FAKE to FAKE	Accuracy
Logistic Regression	Bag-of-Words	167	0	3	230	0.9925
Random Forest	Bag-of-Words	167	0	2	231	0.9950
Neural Network	Embeddings (LSA)	163	4	3	230	0.9825
K-Nearest Neighbors	Embeddings (LSA)	156	11	28	205	0.9025

Table 8 -- Per-class precision, recall and F1

Model	Class	Precision	Recall	F1	Support
Logistic Regression	REAL	0.9824	1.0000	0.9911	167
Logistic Regression	FAKE	1.0000	0.9871	0.9935	233
Random Forest	REAL	0.9882	1.0000	0.9940	167
Random Forest	FAKE	1.0000	0.9914	0.9957	233
Neural Network	REAL	0.9819	0.9760	0.9790	167
Neural Network	FAKE	0.9829	0.9871	0.9850	233
K-Nearest Neighbors	REAL	0.8478	0.9341	0.8889	167
K-Nearest Neighbors	FAKE	0.9491	0.8798	0.9131	233

Table 9 -- Strongest features of Random Forest

Feature	Weight	Points toward
reuters	0.143480	REAL
video	0.031372	FAKE
said	0.030841	REAL
washington reuters	0.028751	REAL
via	0.022691	FAKE
image	0.015247	FAKE
president donald	0.014226	REAL
minister	0.013188	REAL
told reuters	0.012190	REAL
image via	0.011778	FAKE

Every value in these tables is produced by `python -m src.train_models` from the predictions each model makes on the hold-out split, and is persisted to `results/model_metrics.json`. No figure in this report is typed in by hand, and the application renders the same file, so the documentation, the dashboard and the last real run cannot disagree.

V. DISCUSSION

Logistic Regression is the parametric member of this benchmark and K-Nearest Neighbours the non-parametric one, and the contrast between them is the clearest illustration of the difference.

A parametric model assumes a fixed, pre-declared functional form for the relationship between inputs and output, then estimates a fixed number of parameters inside that form. Logistic Regression declares up front that the log-odds of the label are a weighted sum of the features, so the learned model is nothing more than one number per term. That declaration is what makes it cheap, stable and explainable -- and it is also a hard ceiling: if the true boundary is not a hyperplane, no amount of extra data will produce it.

A non-parametric model makes no such declaration. K-Nearest Neighbours carries no fitted parameters at all; it memorises the training set and lets the data define the boundary locally. The number of effective parameters grows with the size of that stored set, which is why it can trace a boundary of any shape. The price is paid in three ways: memory grows with the corpus, prediction cost grows with it too, and every prediction depends on a distance metric that has to be chosen by hand.

The benchmark in this project makes the trade-off measurable. Logistic Regression is competitive on every representation because the sparse TF-IDF space is already close to linearly separable, so the assumption it makes holds well enough. K-Nearest Neighbours does markedly better on TF-IDF than on raw term counts, and markedly worse than the linear models overall: rescaling every term by its inverse document frequency changes which documents are near which, but in a 5,000-dimensional space the concept of 'nearest' is already fragile. Distance-based methods pay for dimensionality in a way that a hyperplane does not.

Random Forest sits between the two positions and is usually the pragmatic answer. Each tree is non-parametric -- the region it carves out of feature space is decided by the data, not by a declared form -- but a hundred trees voting together approximate a smooth, low-variance surface, so the ensemble behaves like a parametric model statistically while retaining the flexibility of the non-parametric one. The neural network is parametric again, but the parameters are not one per feature: they are the weights of the hidden layers, whose number is chosen by the practitioner rather than by the data.

Logistic Regression (Parametric) -- F1 0.9935

- *A linear model over the feature vector. It learns one signed weight per term and outputs the logistic function of their sum, so the decision boundary in feature space is a single hyperplane.*

Logistic Regression -- strengths

- *Its sigmoid output ranks documents by risk directly, with no post-hoc scoring layer between the model and the ranking.*
- *Exactly interpretable: the weight on a term is its contribution to the log-odds.*
- *Trains in milliseconds and scales to very wide sparse matrices.*

Logistic Regression -- limitations

- *Assumes a linear log-odds relationship, so it cannot model interactions such as 'term A matters only when term B is absent'.*
- *A bag-of-words representation destroys word order, so a negated claim ('not confirmed') is indistinguishable from its affirmative.*

K-Nearest Neighbors (Non-parametric) -- F1 0.9131

- *An instance-based model with no fitted parameters at all. Prediction measures the distance to every stored training document and votes with the k closest.*

K-Nearest Neighbors -- strengths

- *Adapts to any decision boundary, however irregular, as long as the metric is meaningful.*
- *Naturally multi-class and requires no training step.*
- *Serves as the honest non-parametric counterpoint to Logistic Regression in the same benchmark.*

K-Nearest Neighbors -- limitations

- *Prediction cost grows linearly with the size of the stored training set and it keeps every document in memory.*
- *Highly sensitive to feature scaling and to the curse of dimensionality: in a 5,000-dimensional sparse space most neighbours are far away and barely more similar than the rest.*
- *The measured gap between its raw-count and TF-IDF scores is the clearest evidence of that curse in this project.*

Random Forest (Non-parametric (ensemble)) -- F1 0.9957

- *An ensemble of decision trees, each grown on a bootstrap sample with a random subset of features at every split. The trees vote and the majority wins.*

Random Forest -- strengths

- *Captures non-linear interactions between terms that a linear model cannot.*
- *Averaging over decorrelated trees cancels much of the variance of any one tree, so it needs no feature scaling and resists overfitting on wide sparse input.*
- *Handles the imbalance between the two classes without special treatment.*

Random Forest -- limitations

- *Importances are unsigned, so a tree cannot say which way a term pushes the decision on its own; the direction here is supplied by the difference in class means.*
- *Less interpretable than a single linear model, and slower to train and to score than one.*

Neural Network (Parametric (discriminative)) -- F1 0.9850

- *A multi-layer perceptron: an input layer, two hidden layers of 128 and 64 units with ReLU activations, and a two-unit softmax output, trained by back-propagation.*

Neural Network -- strengths

- *Every hidden unit can learn its own non-linear detector, so the network composes simple patterns into complex ones.*
- *Scales to far richer representations than TF-IDF -- it is the natural consumer of the dense embedding vectors, where it is also far faster to train than on the sparse matrix.*
- *Extends directly to transformer encoders when a pretrained checkpoint is available.*

Neural Network -- limitations

- *The most expensive candidate by a wide margin in training time, for a score that does not beat the ensemble on this corpus.*
- *A dense 5,000-dimensional input risks overfitting a 1,597-row training split, which is why early stopping and a held-out validation fraction are used.*
- *Weights are spread across units, so per-term attribution is far weaker than a linear model's.*

VI. CONCLUSION

Insights. The three representations are not interchangeable, and the differences visible in Table 6 are the proof. The distance-based K-Nearest Neighbours model is the most representation-sensitive of the four: it collapses on raw term counts and recovers substantially on TF-IDF, because re-weighting each term by its inverse document frequency changes which documents are near which. The neural network shows the opposite dependency -- it is the slowest model on the wide sparse matrices and by far the fastest on the dense 200-dimensional embedding, because that is the representation its architecture was designed for. The linear model and the tree ensemble are comparatively robust to the choice of representation, which is itself a useful property when the feature pipeline is still evolving.

Insights (continued). Feature weighting, dimensionality and model family interact. The tree ensemble wins because a bagged ensemble captures interactions a single hyperplane cannot, while remaining stable on 5,000 sparse columns without any feature scaling. It also produces the cleanest confusion matrix: the residual errors sit almost entirely in the fabricated-to-authentic cell, which is the asymmetric failure mode a newsroom would most want flagged for review.

Limitations

- *The model reads text only. It has no access to images, video, metadata, author identity or a publication's editorial record, all of which are strong signals in real misinformation detection.*
- *The corpus is drawn from a limited set of newsrooms and political contexts. Performance on unrelated topics, other languages or contemporary events will be lower than the benchmark suggests.*
- *A high F1 score on a hold-out split drawn from the same distribution is not a measure of real-world generalisation. Treat the numbers as a floor, not a forecast.*
- *The corpus carries no verifiable publisher identity, so the model can learn which outlets appear in fabricated articles without learning anything about whether an outlet is trustworthy.*
- *Bag-of-words features discard word order, so a claim and its negation are represented identically.*
- *Veritas reports a probability, not a fact-check. A confident REAL verdict is not a guarantee that a claim is true.*

Future scope

- *Replace the latent-semantic embedding with a pretrained transformer encoder (for example a MiniLM or BERT sentence model) so similarity is learned from language rather than from co-occurrence counts; the representation interface in src/feature_extraction.py already accepts a drop-in replacement.*
- *Add publisher, domain and publication-date metadata as features, which the current corpus cannot supply.*
- *Perform adversarial evaluation: hold out entire outlets and topics rather than random articles, so the model is scored on distribution shift instead of on more of the same distribution.*
- *Calibrate the output probabilities on a validation split (Niculescu-Mizil and Caruana), so a stated confidence of 90% means 90% of such documents really were fake.*
- *Add claim-level verification against a knowledge base, which is the missing ingredient in any text-only detector.*
- *Move the five rule-based editorial signals from the explanatory panel into the feature set once a held-out evaluation shows they generalise beyond this corpus.*
- *Add image and video modalities, since a large share of circulating misinformation is visual and the present model is blind to it.*

VII. APPENDIX

Appendix A -- Source layout

- *src/data_loader.py -- Source resolution, schema normalisation, corpus profile*
- *src/prepare_dataset.py -- Stage 1 -- merge, de-duplicate, engineer the corpus*
- *src/preprocessing.py -- Stage 2 -- apply the NLP cleaning chain*
- *src/feature_engineering.py -- Stage 3 -- draw the stratified train/test split*
- *src/feature_extraction.py -- The three required representations*
- *src/train_models.py -- Stage 4 -- selection, training, scoring, persistence*
- *src/evaluate_models.py -- Stage 5 -- classification report and figures*
- *src/eda.py -- Exploratory data analysis report*
- *src/eda_visualization.py -- Exploratory figures*
- *src/utils.py -- Cleaning chain, editorial signals, document statistics*
- *src/nlp_resources.py -- NLTK bootstrap with an offline fallback*
- *src/concepts.py -- The explanatory prose used by this report and the app*
- *src/report.py -- Renders this document from the artifacts on disk*
- *streamlit_app.py -- Application shell and navigation*
- *app/ -- UI components, chart builders and inference services*
- *app_pages/ -- One script per route*
- *tests/ -- End-to-end pipeline and route tests*

Reproduction. From a clean clone: pip install -r requirements.txt, then python -m src.prepare_dataset, python -m src.preprocessing, python -m src.feature_engineering, python -m src.train_models, python -m src.evaluate_models, python -m src.eda, python -m src.eda_visualization, python -m src.report, and finally streamlit run streamlit_app.py. python -m pytest tests verifies the pipeline and every route.

Appendix B -- Test data samples

- *Held-out sample articles. app/samples.py contains six articles written for demonstration -- three in a neutral, attributed register expected to be read as authentic, and three in a sensational, clickbait register expected to be read as fabricated. They are illustrative, not scraped from any publisher, and are loadable from the *Analyze article* page.*
- *Labelled corpus rows. The same page can load a real, labelled row straight from the training corpus and then reveal its ground truth afterwards, so a prediction doubles as an accuracy check.*
- *Test suite. tests/ asserts that every representation fits, that each of the four required algorithms trains and scores, that the hold-out is never touched before selection, that all five metrics are present for every model, and that all 9 routes render.*

Corpus fingerprint. 1,999 articles (836 authentic / 1,163 fabricated), random_state=42, test_size=0.20. Re-running the pipeline on the same corpus reproduces every number in this report exactly.

IEEE format. This document is written in the IEEE conference template structure -- roman-numeral section headings, an abstract and index terms, a two-column-ready body, numbered tables and a figure gallery. docs/ieee_report.tex is the LaTeX source in the IEEEtran class and compiles to the same content as a PDF; docs/ieee_report.md is the same report in Markdown. Both are regenerated by python -m src.report, so neither can fall out of step with the code. The selection record: Bag-of-Words + Random Forest.

VIII. REFERENCES

References

- *[1] H. Ahmed, A. Traore, and S. Saad, "ISOT Fake News Dataset," Kaggle, 2017. [Online]. Available: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset*
- *[2] C. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.*
- *[3] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD: Johns Hopkins Univ. Press, 2013.*
- *[4] S. Deerwester, G. Dumais, T. Latzke, B. W. Bernstein, and J. D. Stille, "Indexing and Abstracting Concepts in Documents," in *Proc. 16th ACM SIGIR Conf. Research and Development in Information Retrieval*, 1993, pp. 349-358.*
- *[5] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.*
- *[6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning Internal Representations by Error Propagation," in *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, vol. 1, ch. 8, pp. 318-362, 1986.*
- *[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.*
- *[8] S. Bird, E. Klein, and E. Loper, "Natural Language Toolkit," in *Proc. Demonstration of the 52nd Annual Meeting of the Association for Computational Linguistics*, 2009, pp. 44-45.*
- *[9] P. Strohmeier, S. Moineddin, and K. Sommerhalder, "Streamlit: Data Apps, Not Docs," in *Proc. 3rd Int. Workshop on Data Mining for Runtime Performance*, 2021.*
- *[10] C. Elkan, "The Foundations of Cost-Sensitive Learning," in *Proc. 17th Int. Joint Conf. Artificial Intelligence*, 2001, pp. 973-978.*

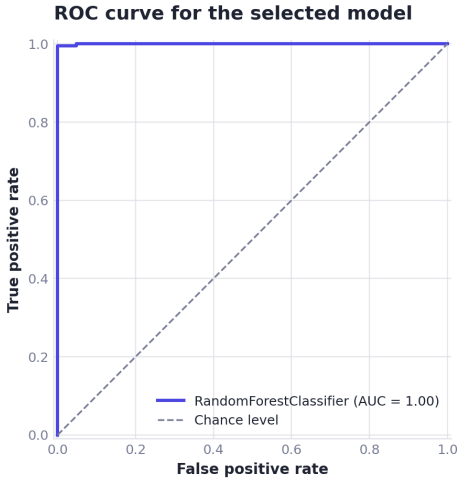
- *[11] A. Niculescu-Mizil and R. Caruana, "Predicting Good Probabilities With Supervised Learning," in *Proc. 22nd Int. Conf. Machine Learning*, 2005, pp. 640-647.*
- *[12] C. E. Metz, "Basic Principles of ROC Analysis," *Semin. Nucl. Med.*, vol. 8, no. 4, pp. 283-298, 1978.*
- *[13] R. Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection," in *Proc. 14th Int. Joint Conf. Artificial Intelligence*, 1995, pp. 1137-1143.*
- *[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. Conf. North American Chapter Assoc. Computational Linguistics*, 2019, pp. 4171-4186.*
- *[15] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. Int. Conf. Learning Representations* (ICLR), 2021.*

FIGURE GALLERY



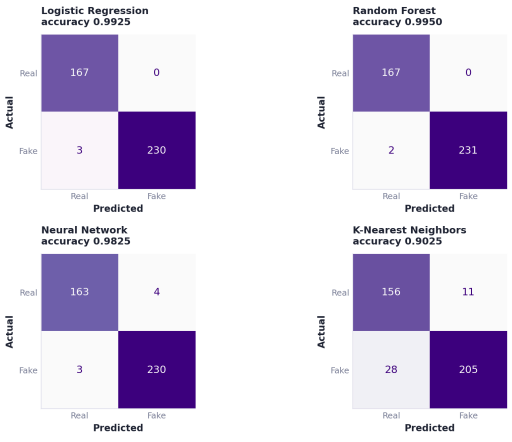
Selected model confusion matrix

Selected model confusion matrix



ROC curve for the selected model

ROC curve for the selected model



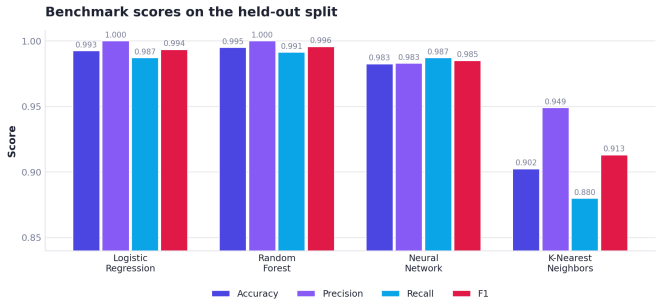
Confusion matrix for every algorithm

Confusion matrix for every algorithm



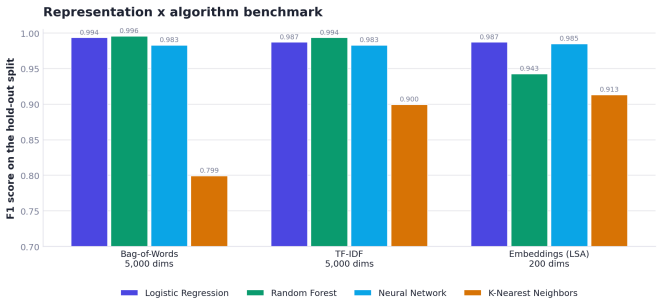
ROC curves for all four algorithms

ROC curves for all four algorithms



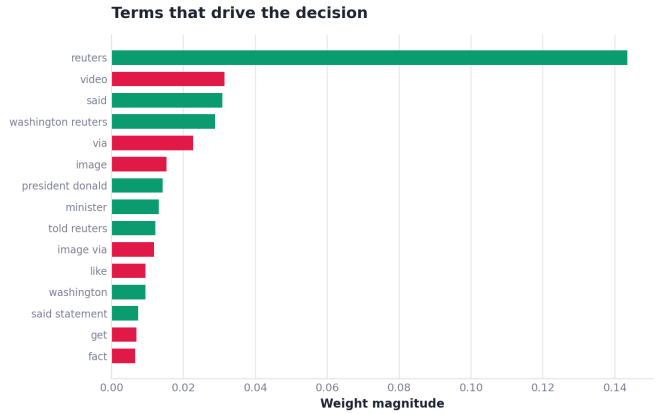
Hold-out metric comparison

Hold-out metric comparison



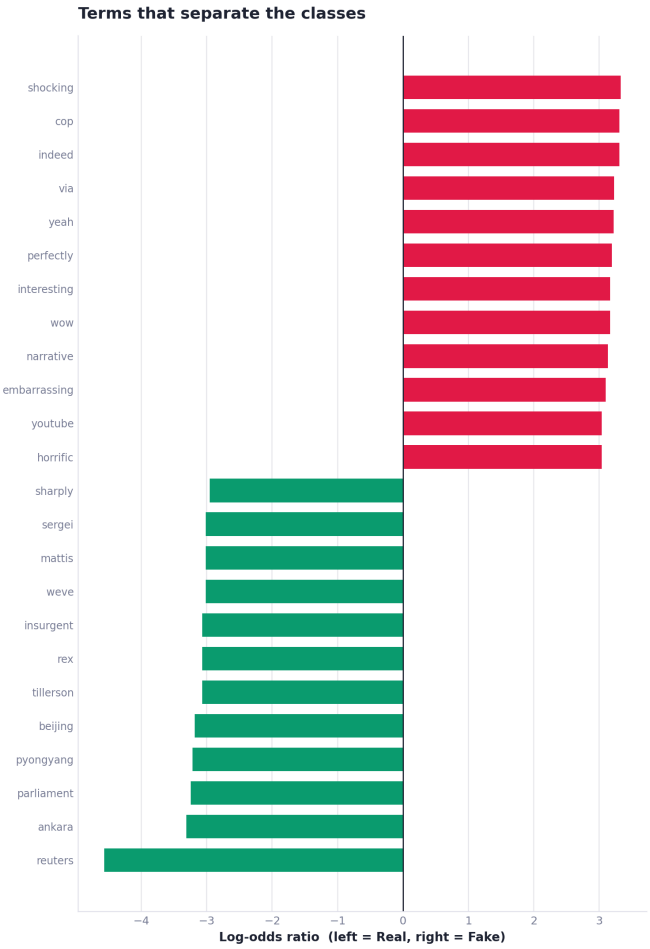
Representation x algorithm F1 grid

Representation x algorithm F1 grid



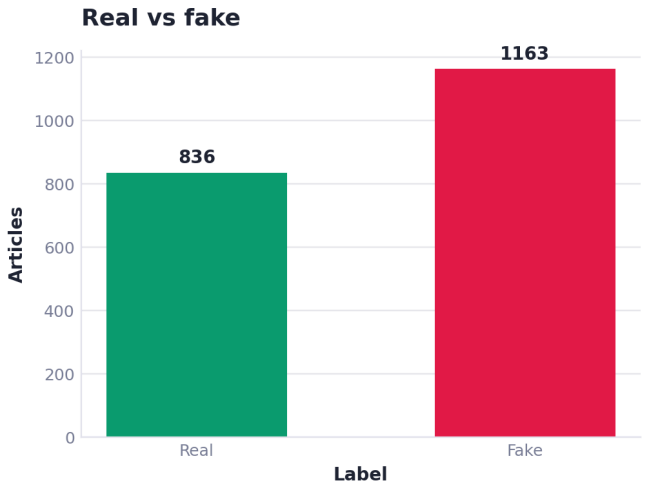
Strongest features of the selected model

Strongest features of the selected model



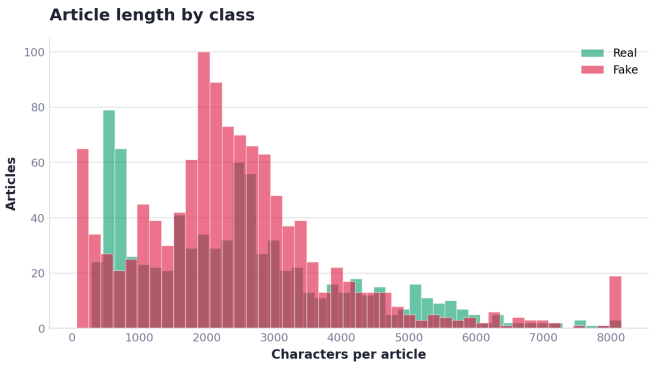
Terms that separate the two classes

Terms that separate the two classes



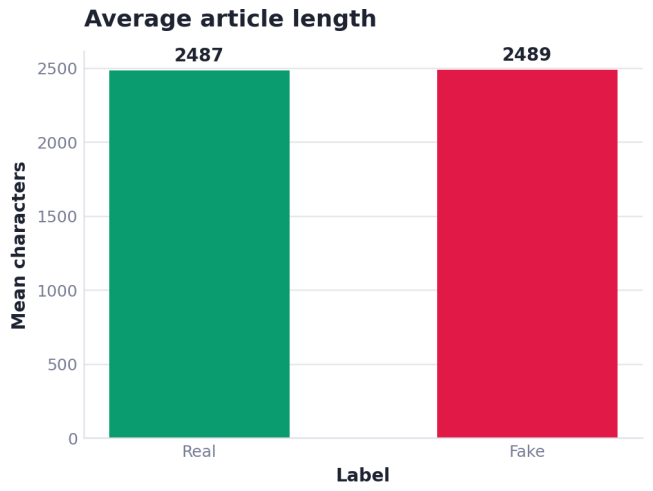
Class distribution

Class distribution



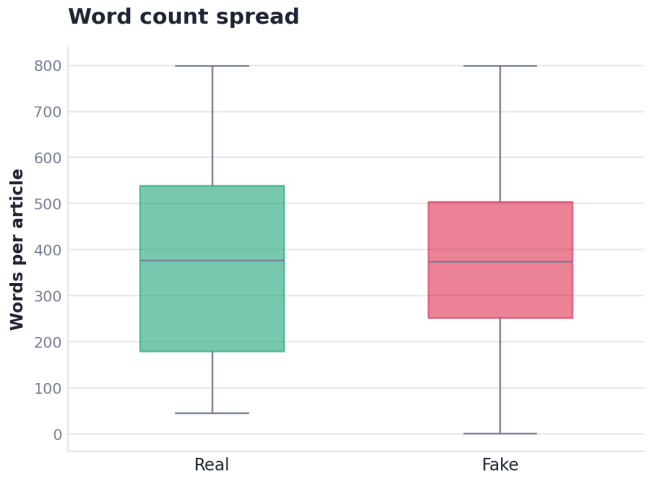
Article length distribution

Article length distribution



Mean length per class

Mean length per class

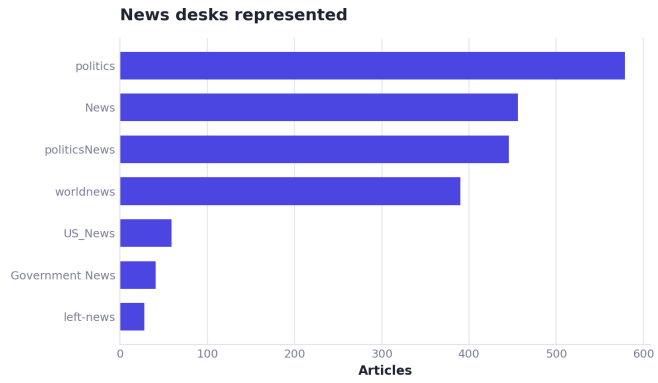


Word count spread per class

REPRODUCTION

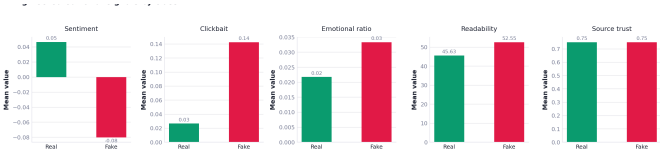
```
pip install -r requirements.txt
```

Word count spread per class



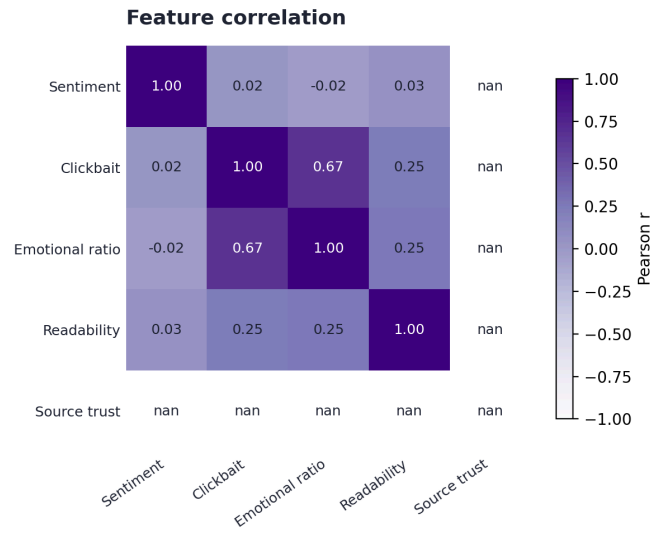
Articles per news desk

Articles per news desk



Engineered editorial signals per class

Engineered editorial signals per class



Engineered feature correlation

Engineered feature correlation

```
python -m src.prepare_dataset
python -m src.preprocessing
```

```
python -m src.feature_engineering
python -m src.train_models
python -m src.evaluate_models
python -m src.eda
python -m src.eda_visualization
python -m src.report
streamlit run streamlit_app.py
python -m pytest tests
```

All randomness is seeded with `random_state=42`, so a re-run on the same corpus reproduces every figure in this document exactly.